

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

Aim :- Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

CODE :-

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
```

```
# Step 1: Generate RSA Key Pair
```

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
```

```
public_key = private_key.public_key()
```

```
print("RSA Key Pair Generated Successfully!")
```

```
# Step 2: Take Message Input
```

```
message = input("Enter message: ").encode()
```

```
# Step 3: Generate Digital Signature
```

```
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
```

```
print("\nDigital Signature Generated Successfully!")
```

```
print("Digital Signature:", signature.hex())
```

```
# Step 4: Verify Original Message
```

```
try:
```

```
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),

```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
        hashes.SHA256()
    )

    print("\nSignature Verified!")
    print("Message Integrity: Maintained")
    print("Message Authenticity: Verified")

except Exception:
    print("\nSignature Verification Failed!")

# Step 5: Test Tampered Message
tampered_message = input(
    "\nEnter message for verification: "
).encode()

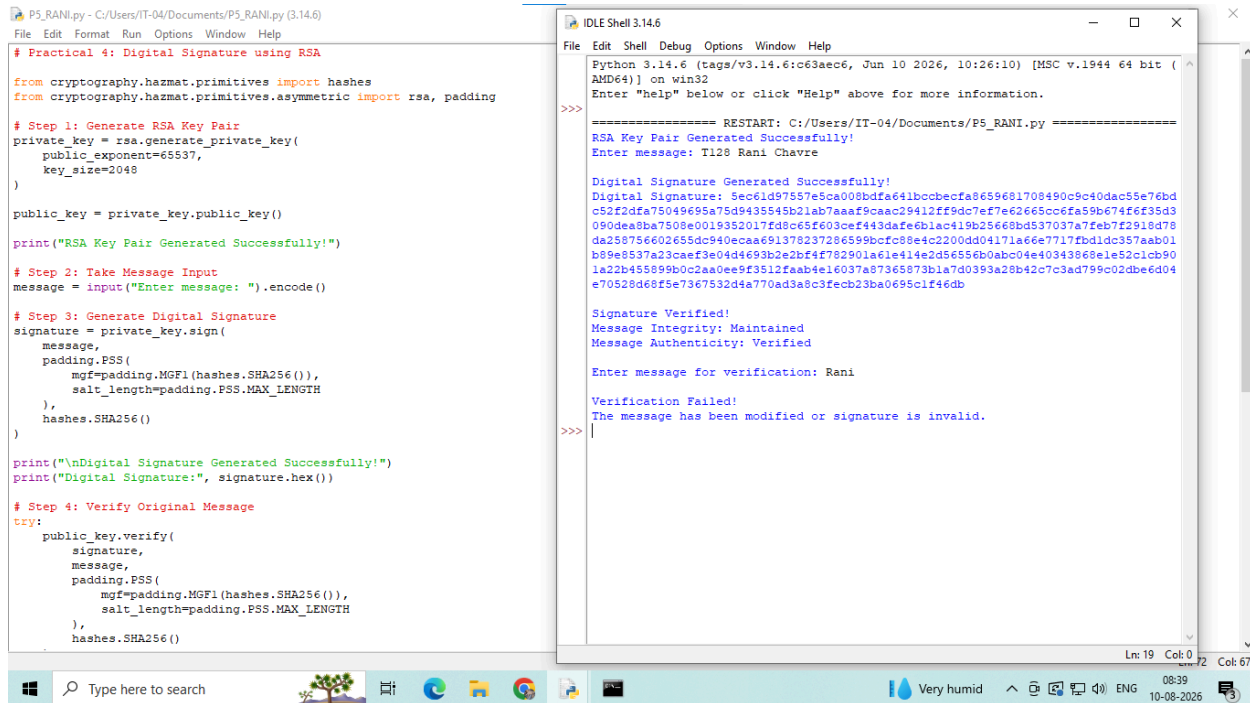
try:
    public_key.verify(
        signature,
        tampered_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("\nSignature Verified!")
    print("Message is Authentic")

except Exception:
    print("\nVerification Failed!")
    print("The message has been modified or signature is invalid.")
```

SHETH L.U.J AND SIR MV COLLEGE

PRACTICAL NO:- 04



```
# Practical 4: Digital Signature using RSA
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding

# Step 1: Generate RSA Key Pair
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

public_key = private_key.public_key()

print("RSA Key Pair Generated Successfully!")

# Step 2: Take Message Input
message = input("Enter message: ").encode()

# Step 3: Generate Digital Signature
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nDigital Signature Generated Successfully!")
print("Digital Signature:", signature.hex())

# Step 4: Verify Original Message
try:
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
except:
```

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

===== RESTART: C:/Users/II-04/Documents/P5_RANI.py =====
RSA Key Pair Generated Successfully!
Enter message: Tl28 Rani Chavre

Digital Signature Generated Successfully!
Digital Signature: 5ec61d97557e5ca008bdfa641b0cbeafa8659601708490c9e40dac55e76bd
c52f2dfa75049695a75d9435545b21ab7aaaf9caac29412ff9dc7ef7e62665cc6fa59b674f6f35d3
090dea8ba7508e0019352017fd8c65f603cef443dafa6blac419b25668bd537037a7feb7f2918d78
da258756602655dc940ccaa691378237286599b0cfc88e4c2200dd04171a66e7717fbd1dc357aab01
b89e8537a23caef3e04d4693b2e2bf4f782901a61e414e2d56556b0abc04e40343868e1e52c1cb90
1a22b455899b0c2aa0ee9f3512faab4e16037a87365873b1a7d0393a28b42c7c3ad799c02dbe6d04
e70528d68f5e7367532d4a770ad3a8c3fecb23ba0695c1f4edb

Signature Verified!
Message Integrity: Maintained
Message Authenticity: Verified

Enter message for verification: Rani

Verification Failed!
The message has been modified or signature is invalid.
```

Practical: RSA Digital Signature - CLI

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
```

```
# Step 1: Generate RSA Key Pair
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
```

```
public_key = private_key.public_key()
```

```
print("=" * 60)
print("      RSA DIGITAL SIGNATURE")
print("=" * 60)
```

```
print("\n[✓] RSA Key Pair Generated Successfully!")
```

```
# Step 2: Take Message Input
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
message = input("\nEnter message: ").encode()

# Step 3: Generate Digital Signature
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\n[✓] Digital Signature Generated Successfully!")
print("\nDigital Signature:")
print(signature.hex())

# Step 4: Verify Original Message
try:
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("\n" + "=" * 60)
    print("SIGNATURE VERIFICATION")
    print("=" * 60)

    print("\n[✓] Signature Verified!")
    print("[✓] Message Integrity : Maintained")
    print("[✓] Message Authenticity: Verified")

except Exception:
    print("\n[X] Signature Verification Failed!")

# Step 5: Test Tampered Message
tampered_message = input(
```

SHETH L.U.J AND SIR MV COLLEGE

PRACTICAL NO:- 04

```
"\nEnter message for verification: "
).encode()
```

try:

```
public_key.verify(
    signature,
    tampered_message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
```

```
print("\n[✓] Signature Verified!")
print("[✓] Message is Authentic")
```

except Exception:

```
print("\n[X] Verification Failed!")
print("[!] The message has been modified or signature is invalid.")
```

The image shows a screenshot of a computer screen with two windows. The left window is a text editor showing a Python script for RSA digital signature verification. The script includes functions for generating a key pair, signing a message, and verifying a signature. It also includes a test case for a tampered message. The right window is the IDLE Shell, showing the execution of the script. The output displays the successful generation of an RSA key pair, the signing of the message 'Rani Chavre', and the verification of the signature, which is confirmed as authentic. The script also demonstrates that a tampered message fails verification.

```
mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH
),
hashes.SHA256()
)

print("\n" + "=" * 60)
print("SIGNATURE VERIFICATION")
print("=" * 60)

print("\n[✓] Signature Verified!")
print("[✓] Message Integrity : Maintained")
print("[✓] Message Authenticity: Verified")

except Exception:
    print("\n[X] Signature Verification Failed!")

# Step 5: Test Tampered Message
tampered_message = input(
    "\nEnter message for verification: "
).encode()

try:
    public_key.verify(
        signature,
        tampered_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("\n[✓] Signature Verified!")
    print("[✓] Message is Authentic")

except Exception:
    print("\n[X] Verification Failed!")
    print("[!] The message has been modified or signature is invalid.")
```

```
Python 3.14.6 (tags/v3.14.6:063a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/IT-04/Documents/rani_P4_CLI_cyber.py =====
RSA DIGITAL SIGNATURE

[✓] RSA Key Pair Generated Successfully!

Enter message: Rani Chavre

[✓] Digital Signature Generated Successfully!

Digital Signature:
3014708fd44793ffb12e6a34492f0a5a624e13a22b8c11e1a3fd87c498fac6810eb7c0dc495a097a
d0da01735d9e758ca9cd6678d098e993f5827f5795e144205427a085fd9d0cdee6ec6178c5c54003
4cfd57d9cd2d9e98c9ab537375e5a879c2c03c4d30d1ab210cc26923d098db0f256bb94e175394
b12923de8d241be3e25d6c2feff8dacbda8f1b41cac14015d8d9a97bb13f8e6e435bb406f752f5c6
eb4e4b53c4bf09da793c37c17b72311347bbe4eb8d57863055766913c9ff3a7aa152ddc6e8e62711
9b7040e8c4e510e4d1f978db151eb99b8a68e9120553b9a65101ff8aca08573f094ba5e23112361
0d3224d7285ba32eb4bcl82187ed831a

=====
SIGNATURE VERIFICATION

[✓] Signature Verified!
[✓] Message Integrity : Maintained
[✓] Message Authenticity: Verified

Enter message for verification: Rani Chavre

[✓] Signature Verified!
[✓] Message is Authentic

===== RESTART: C:/Users/IT-04/Documents/rani_P4_CLI_cyber.py =====
RSA DIGITAL SIGNATURE
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

Practical: RSA Digital Signature - GUI

```
import tkinter as tk
from tkinter import messagebox
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding

# =====
# RSA KEY GENERATION
# =====

private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

public_key = private_key.public_key()

signature = None
original_message = ""

# =====
# GENERATE DIGITAL SIGNATURE
# =====

def generate_signature():

    global signature
    global original_message

    original_message = message_box.get("1.0", tk.END).strip()

    if original_message == "":
        messagebox.showwarning(
            "Input Required",
            "Please enter a message."
        )
        return

    signature = private_key.sign(
        original_message.encode(),
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
padding.PSS(  
    mgf=padding.MGF1(hashes.SHA256()),  
    salt_length=padding.PSS.MAX_LENGTH  
)  
hashes.SHA256()  
)
```

```
# Display signature  
signature_box.config(state="normal")  
signature_box.delete("1.0", tk.END)  
signature_box.insert(tk.END, signature.hex())  
signature_box.config(state="disabled")
```

```
status_label.config(  
    text="✓ DIGITAL SIGNATURE GENERATED",  
    fg="#16a34a"  
)
```

```
integrity_label.config(  
    text="Integrity: Ready for Verification",  
    fg="#64748b"  
)
```

```
authenticity_label.config(  
    text="Authenticity: Ready for Verification",  
    fg="#64748b"  
)
```

```
# =====  
# VERIFY ORIGINAL MESSAGE  
# =====
```

```
def verify_message():  
  
    if signature is None:  
        messagebox.showwarning(  
            "No Signature",  
            "Please generate the digital signature first."  
        )  
        return  
  
    try:
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
public_key.verify(  
    signature,  
    original_message.encode(),  
    padding.PSS(  
        mgf=padding.MGF1(hashes.SHA256()),  
        salt_length=padding.PSS.MAX_LENGTH  
    ),  
    hashes.SHA256()  
)
```

```
status_label.config(  
    text="✓ SIGNATURE VERIFIED",  
    fg="#16a34a"  
)
```

```
integrity_label.config(  
    text="✓ Message Integrity: Maintained",  
    fg="#16a34a"  
)
```

```
authenticity_label.config(  
    text="✓ Message Authenticity: Verified",  
    fg="#16a34a"  
)
```

except Exception:

```
status_label.config(  
    text="✗ SIGNATURE VERIFICATION FAILED",  
    fg="#dc2626"  
)
```

```
integrity_label.config(  
    text="✗ Message Integrity: Failed",  
    fg="#dc2626"  
)
```

```
authenticity_label.config(  
    text="✗ Message Authenticity: Failed",  
    fg="#dc2626"  
)
```

=====

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

TAMPERED MESSAGE TEST

=====

def verify_tampered():

```
    if signature is None:
        messagebox.showwarning(
            "No Signature",
            "Please generate the digital signature first."
        )
    return
```

```
test_message = tampered_box.get("1.0", tk.END).strip()
```

```
if test_message == "":
    messagebox.showwarning(
        "Input Required",
        "Please enter a message for testing."
    )
    return
```

try:

```
    public_key.verify(
        signature,
        test_message.encode(),
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
```

```
    tamper_status.config(
        text="✓ Signature Verified - Message is Authentic",
        fg="#16a34a"
    )
```

except Exception:

```
    tamper_status.config(
        text="✗ Verification Failed - Message has been Modified!",
        fg="#dc2626"
    )
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
# =====
# MAIN WINDOW
# =====

root = tk.Tk()

root.title("RSA Digital Signature")
root.geometry("900x800")
root.resizable(False, False)

root.configure(bg="#eef2f7")

# =====
# HEADER
# =====

header = tk.Frame(
    root,
    bg="#1e293b",
    height=105
)

header.pack(fill="x")

title = tk.Label(
    header,
    text="🔒 RSA DIGITAL SIGNATURE",
    font=("Segoe UI", 25, "bold"),
    bg="#1e293b",
    fg="white"
)

title.pack(pady=(20, 3))

subtitle = tk.Label(
    header,
    text="Secure Message Integrity & Authenticity Verification",
    font=("Segoe UI", 11),
    bg="#1e293b",
    fg="#cbd5e1"
)
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
subtitle.pack()
```

```
# =====  
# RSA KEY CARD  
# =====
```

```
key_card = tk.Frame(  
    root,  
    bg="white",  
    highlightbackground="#d7dee8",  
    highlightthickness=1  
)
```

```
key_card.pack(  
    padx=35,  
    pady=18,  
    fill="x"  
)
```

```
key_title = tk.Label(  
    key_card,  
    text="🔑 RSA KEY PAIR",  
    font=("Segoe UI", 12, "bold"),  
    bg="white",  
    fg="#334155"  
)
```

```
key_title.pack(side="left", padx=20, pady=15)
```

```
key_status = tk.Label(  
    key_card,  
    text="● 2048-bit RSA Key Generated",  
    font=("Segoe UI", 11, "bold"),  
    bg="white",  
    fg="#16a34a"  
)
```

```
key_status.pack(side="right", padx=20)
```

```
# =====  
# ORIGINAL MESSAGE
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

=====

```
message_title = tk.Label(
    root,
    text="ORIGINAL MESSAGE",
    font=("Segoe UI", 11, "bold"),
    bg="#eef2f7",
    fg="#475569"
)
```

```
message_title.pack(
    anchor="w",
    padx=35,
    pady=(0, 5)
)
```

```
message_box = tk.Text(
    root,
    height=3,
    font=("Segoe UI", 12),
    bg="white",
    fg="#1e293b",
    relief="flat",
    highlightbackground="#d7dee8",
    highlightthickness=1,
    wrap="word"
)
```

```
message_box.pack(
    padx=35,
    fill="x"
)
```

=====

GENERATE BUTTON

=====

```
generate_button = tk.Button(
    root,
    text="◆ GENERATE DIGITAL SIGNATURE",
    font=("Segoe UI", 11, "bold"),
    bg="#2563eb",
    fg="white",
)
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
        activebackground="#1d4ed8",
        activeforeground="white",
        relief="flat",
        cursor="hand2",
        command=generate_signature
    )

generate_button.pack(
    pady=15,
    padx=20,
    ipady=9
)

# =====
# DIGITAL SIGNATURE
# =====

signature_title = tk.Label(
    root,
    text="DIGITAL SIGNATURE",
    font=("Segoe UI", 11, "bold"),
    bg="#eef2f7",
    fg="#475569"
)

signature_title.pack(
    anchor="w",
    padx=35,
    pady=(0, 5)
)

signature_box = tk.Text(
    root,
    height=5,
    font=("Consolas", 9),
    bg="white",
    fg="#2563eb",
    relief="flat",
    highlightbackground="#d7dee8",
    highlightthickness=1,
    wrap="word"
)
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
signature_box.pack(
    padx=35,
    fill="x"
)

signature_box.config(state="disabled")

# =====
# VERIFY BUTTON
# =====

verify_button = tk.Button(
    root,
    text="✓ VERIFY ORIGINAL MESSAGE",
    font=("Segoe UI", 11, "bold"),
    bg="#16a34a",
    fg="white",
    activebackground="#15803d",
    activeforeground="white",
    relief="flat",
    cursor="hand2",
    command=verify_message
)

verify_button.pack(
    pady=12,
    ipadx=20,
    ipady=8
)

# =====
# VERIFICATION STATUS
# =====

status_card = tk.Frame(
    root,
    bg="white",
    highlightbackground="#d7dee8",
    highlightthickness=1
)

status_card.pack()
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
        padx=35,
        fill="x"
    )

    status_label = tk.Label(
        status_card,
        text="● WAITING FOR VERIFICATION",
        font=("Segoe UI", 12, "bold"),
        bg="white",
        fg="#64748b"
    )

    status_label.pack(pady=(12, 5))

    integrity_label = tk.Label(
        status_card,
        text="Integrity: Not Verified",
        font=("Consolas", 10),
        bg="white",
        fg="#64748b"
    )

    integrity_label.pack()

    authenticity_label = tk.Label(
        status_card,
        text="Authenticity: Not Verified",
        font=("Consolas", 10),
        bg="white",
        fg="#64748b"
    )

    authenticity_label.pack(pady=(0, 12))

    # =====
    # TAMPERED MESSAGE TEST
    # =====

    tamper_title = tk.Label(
        root,
        text="TAMPERED MESSAGE TEST",
        font=("Segoe UI", 11, "bold"),
        bg="#eef2f7",
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
fg="#475569"
)

tamper_title.pack(
    anchor="w",
    padx=35,
    pady=(15, 5)
)

tamper_frame = tk.Frame(
    root,
    bg="white"
)

tamper_frame.pack(
    padx=35,
    fill="x"
)

tampered_box = tk.Text(
    tamper_frame,
    height=2,
    font=("Segoe UI", 11),
    bg="#f8fafc",
    fg="#1e293b",
    relief="flat",
    wrap="word"
)

tampered_box.pack(
    side="left",
    padx=12,
    pady=10,
    fill="both",
    expand=True
)

tamper_button = tk.Button(
    tamper_frame,
    text="TEST",
    font=("Segoe UI", 10, "bold"),
    bg="#f59e0b",
    fg="white",
    activebackground="#d97706",
```

SHETH L.U.J AND SIR MV COLLEGE
PRACTICAL NO:- 04

```
        relief="flat",
        cursor="hand2",
        command=verify_tampered
    )

    tamper_button.pack(
        side="right",
        padx=12,
        ipadx=15,
        ipady=7
    )

    tamper_status = tk.Label(
        root,
        text="Enter a message and click TEST",
        font=("Segoe UI", 9),
        bg="#eef2f7",
        fg="#64748b"
    )

    tamper_status.pack(pady=5)

# =====
# FOOTER
# =====

footer = tk.Label(
    root,
    text="Cybersecurity & Authentication | RSA-2048 | SHA-256 | PSS",
    font=("Segoe UI", 8),
    bg="#eef2f7",
    fg="#94a3b8"
)

footer.pack(
    side="bottom",
    pady=5
)

# =====
# START GUI
# =====
```

SHETH L.U.J AND SIR MV COLLEGE

PRACTICAL NO:- 04

root.mainloop()

